Code: https://github.com/zhurleonid/corporate-risk-management
(`homework_3/part2_var_backtest/hw3_part2_code.ipynb`)

Homework 3 comes as three problem sheets; this is sheet 2, numbered Q1 to Q4 as on the sheet. The write-up is below. The code is appended at the end. The derivations, Q2 (the loss operator), Q3 (the VaR formula under normal returns) and Q4 (the confidence interval), are handwritten and attached separately; the write-up points to them where they belong.

A note on the problem sheet

The sheet sets $J = N - M$ with N the number of observations. $N = 2515$ closing prices give 2514 log returns, and the window and the evaluation days are both counted in returns, so I use $J = 2514 - 1010 = 1504$. The one-day difference does not change any conclusion.

VaR backtesting on the S&P 500

Q1. Data

Daily closes of the S&P 500 index (ticker `^GSPC`) from Yahoo Finance, 2016-05-23 through 2026-05-22, saved to `sp500_daily.csv` .

Date	Close
2016-05-23	2048.04
2016-05-24	2076.06
2016-05-25	2090.54
2016-05-26	2090.10
2016-05-27	2099.06
2026-05-18	7403.05
2026-05-19	7353.61
2026-05-20	7432.97
2026-05-21	7445.72
2026-05-22	7473.47

First and last five closes. $N = 2515$ observations, giving 2514 log returns.

With a window of $M = 1010$ returns, the first VaR is produced after the 1010th return and tested on the 1011th, so there are $J = 2514 - 1010 = 1504$ evaluation days, from 2020-05-29 to 2026-05-22.

Q2. Historical simulation on a rolling window

> **Q2: the loss operator for the $1 position by hand, attached.**

Pseudocode

```
x_1 ... x_n  the 2514 log returns,  M = 1010,  k =
ceil(M*alpha) = 960
loss(x) = -(exp(x) - 1)                    $1 position, full
operator
for s = M ... n-1:                         J = n - M days
    window  = x_{s-M+1} ... x_s            last M returns,
through day s
    losses  = loss(window)
    VaR_s   = sorted losses at position k
    L_{s+1} = loss(x_{s+1})                realized loss
next day
    I_{s+1} = 1 if L_{s+1} > VaR_s else 0
N_hat = sum of the I's
```

Result

55 exceedances in 1504 days, a 3.66% rate against the 5% the model promises.

Q3. Normal log returns with EWMA updating

> **Q3: the VaR formula under normal log returns by hand, attached.**

The initial $\hat{\mu}$ and $\hat{\sigma}^2$ are the sample mean and variance of the first 1010 returns. I use them directly for the first forecast and then update with each realized return, so the recursion is

$$\mu_{s+\Delta} = \lambda\mu_s + (1-\lambda)x_s, \qquad \sigma^2_{s+\Delta} = \theta\sigma^2_s + (1-\theta)(x_s-\mu_s)^2,$$

with the variance update again using the mean before it is updated.

Pseudocode

```
mu = mean(x_1 ... x_M),  s2 = var(x_1 ... x_M)    initial
estimates
z  = Phi^{-1}(1 - alpha)
for s = M ... n-1:
    VaR_s   = 1 - exp(mu + sqrt(s2) * z)          forecast
for day s+1
    L_{s+1} = -(exp(x_{s+1}) - 1)
    I_{s+1} = 1 if L_{s+1} > VaR_s else 0
    s2 = theta*s2 + (1 - theta)*(x_{s+1} - mu)^2  then absorb
day s+1
```

```
    mu = lambda*mu + (1 - lambda)*x_{s+1}
N_hat = sum of the I's
```

Result

94 exceedances in 1504 days, a 6.25% rate.

Q4. Confidence interval for the number of exceedances

> **Q4: the construction of the confidence interval by hand, attached.**

Pseudocode

```
E  = J * (1 - alpha)
sd = sqrt(J * alpha * (1 - alpha))
z  = Phi^{-1}(1 - beta/2)
CI = (E - z*sd, E + z*sd)
for each method: reject if N_hat is outside CI
also: exact binomial quantiles at beta/2 and 1 - beta/2, as a
check
```

Results

With $J = 1504$, $\alpha = 0.95$ and $\beta = 0.05$,

$$\mathbb{E}[N] = 1504 \times 0.05 = 75.20, \qquad \sqrt{J\alpha(1-\alpha)} = 8.452, \qquad \mathrm{CI}_{0.95} = 75.20 \pm 1.96 \,\rangle$$

The exact binomial quantiles give $(59, 92)$, the same interval once rounded to whole exceedances.

Method	$\hat{N}$	rate	verdict
Historical simulation, $M = 1010$	55	3.66%	below the interval: reject
Normal, EWMA(0.97)	94	6.25%	above the interval: reject

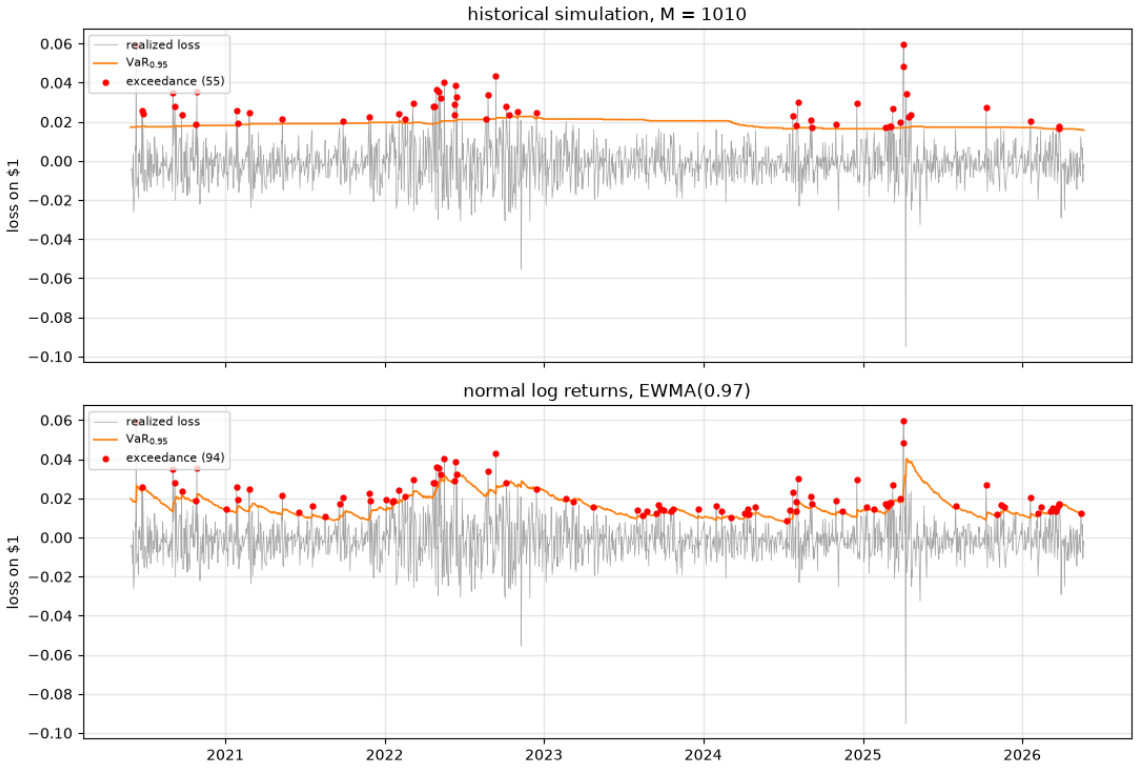
Both models fail, in opposite directions.

Historical simulation is too conservative. The top panel of the figure shows a VaR that barely moves: it sits near 1.9% of the position for four years, then steps down in 2024. The reason is the window. Any day between mid-2020 and early 2024 looks back over March 2020, and the 960th of 1010 ordered losses is set by those crash days whatever the market is doing now. So the VaR was about right for the violent 2022 bear market (20 exceedances that year against 12.5 expected) and far too high for the calm of 2023, where there were none at all. Once the COVID days roll out, the VaR drops and the April

2025 shock produces a cluster of exceedances. The total of 55 is the average of a too-high VaR most of the time and a few clustered misses.

The normal-EWMA model is too aggressive. Its exceedances are spread through every year, 12 to 19 in the full years, which is the 5% rate plus a little, consistently. That consistent excess is the thin tail of the normal distribution: daily index returns have more mass beyond 1.65 standard deviations than a normal does. On top of that, EWMA reacts to a shock only after it has happened, so the first day of every volatility spike is an exceedance by construction; the bottom panel shows the VaR jumping *after* each cluster of red dots. In the lecture's 2007 to 2017 sample the same model gave 79 exceedances out of 1508 and passed. Here it gives 94, and the difference is mostly that this sample has more of its surprises arriving suddenly, in 2020, 2022 and April 2025, rather than building up.

A note on the test itself. The binomial test assumes the exceedances are independent. They are plainly not: 47 of the historical method's 55 land on days when the EWMA model also fails, and both pile up in a few months. A test that also looked at clustering (Christoffersen's independence test, for instance) would reject the historical method more strongly than the count alone suggests.



Realized daily loss on

1 in the S&P 500 over the 1504 evaluation days, with the one-day $\text{VaR}_{0.95}$ *and the exceedances marked. Top: historical simulation on a rolling 1010-day window. Bottom: normal log returns with EWMA(0.97) mean and variance.*

Appendix: source code

The code below produces every figure and number above.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm, binom

ALPHA = 0.95
LAM = 0.97          # EWMA mean
THETA = 0.97        # EWMA variance
M = 1010            # rolling window, in returns

plt.rcParams['figure.figsize'] = (11, 5)
```

In [2]:
```
# Daily S&P 500 closes (^GSPC), saved to csv. To pull fresh data instead:
#     yf.download('^GSPC', start='2016-05-23', end='2026-05-23')['Close']
sp = pd.read_csv('sp500_daily.csv', parse_dates=['Date']).set_index('Date')[
print('first five observations'); print(sp.head())
print('\nlast five observations'); print(sp.tail())
N_SP = len(sp)
print('\nN =', N_SP, 'observations,', sp.index[0].date(), '->', sp.index[-1]

xs = np.log(sp / sp.shift(1)).dropna()
xv = xs.values
J = len(xv) - M
print('log returns:', len(xv), '   window M =', M, '   evaluation periods J
```

```
first five observations
Date
2016-05-23    2048.040039
2016-05-24    2076.060059
2016-05-25    2090.540039
2016-05-26    2090.100098
2016-05-27    2099.060059
Name: Close, dtype: float64

last five observations
Date
2026-05-18    7403.049805
2026-05-19    7353.609863
2026-05-20    7432.970215
2026-05-21    7445.720215
2026-05-22    7473.470215
Name: Close, dtype: float64

N = 2515 observations, 2016-05-23 -> 2026-05-22
log returns: 2514    window M = 1010    evaluation periods J = 1504
```

In [3]:
```
def loss_sp(x):
    # full loss operator for a constant $1 position
    return -(np.exp(x) - 1)
```

```
realized = loss_sp(xv[M:])                  # L_{s+Delta} for the J evaluati
eval_dates = xs.index[M:]

var_h = np.empty(J)
k = int(np.ceil(M * ALPHA))
for j in range(J):
    window = xv[j:j + M]                    # the M returns up to and includ
    var_h[j] = np.sort(loss_sp(window))[k - 1]
exc_h = realized > var_h
print('order statistic k = ceil(M*alpha) =', k)
print('historical simulation: exceedances = %d of %d  (%.2f%%)' % (exc_h.sum
```

```
order statistic k = ceil(M*alpha) = 960
historical simulation: exceedances = 55 of 1504  (3.66%)
```

In [4]:
```
mu_e = xv[:M].mean()                        # initial estimates from the fir
s2_e = xv[:M].var(ddof=1)
print('initial  mu = %.4e   sigma = %.4e' % (mu_e, np.sqrt(s2_e)))

var_e = np.empty(J)
sd_path = np.empty(J)
z = norm.ppf(1 - ALPHA)
for j in range(J):
    var_e[j] = 1 - np.exp(mu_e + np.sqrt(s2_e) * z)      # forecast for the
    sd_path[j] = np.sqrt(s2_e)
    x_new = xv[M + j]                                     # then observe the
    s2_e = THETA * s2_e + (1 - THETA) * (x_new - mu_e)**2
    mu_e = LAM * mu_e + (1 - LAM) * x_new
exc_e = realized > var_e
print('final    mu = %.4e   sigma = %.4e' % (mu_e, np.sqrt(s2_e)))
print('normal + EWMA: exceedances = %d of %d  (%.2f%%)' % (exc_e.sum(), J, 1
```

```
initial  mu = 3.8771e-04   sigma = 1.2413e-02
final    mu = 1.7689e-03   sigma = 8.4225e-03
normal + EWMA: exceedances = 94 of 1504  (6.25%)
```

In [5]:
```
BETA = 0.05
expected = J * (1 - ALPHA)
sd_n = np.sqrt(J * ALPHA * (1 - ALPHA))
zb = norm.ppf(1 - BETA / 2)
lo, hi = expected - zb * sd_n, expected + zb * sd_n
print('J = %d   E[N] = %.2f   sd = %.3f   z = %.4f' % (J, expected, sd_n, zb
print('normal-approximation %.0f%% CI: (%.2f, %.2f)' % (100 * (1 - BETA), lo
print('exact binomial %.0f%% CI:        (%d, %d)' % (100 * (1 - BETA), binom
                                                     binom.ppf(1 - BETA / 2
for name, e in [('historical simulation', exc_h), ('normal + EWMA', exc_e)]:
    inside = lo <= e.sum() <= hi
    print('%-22s N_hat = %3d   %s' % (name, e.sum(), 'inside -> do not rejec
```

```
J = 1504   E[N] = 75.20   sd = 8.452   z = 1.9600
normal-approximation 95% CI: (58.63, 91.77)
exact binomial 95% CI:        (59, 92)
historical simulation  N_hat =  55   OUTSIDE -> reject
normal + EWMA          N_hat =  94   OUTSIDE -> reject
```

In [6]:
```
# yearly breakdown of exceedances, to see where they cluster
yr = pd.DataFrame({'historical': exc_h.astype(int), 'ewma': exc_e.astype(int
by_year = yr.groupby(yr.index.year).sum()
by_year['days'] = yr.groupby(yr.index.year).size()
print(by_year.T)
print('\nmean VaR level: historical %.4f   ewma %.4f' % (var_h.mean(), var_e
print('days both exceed: %d   historical only: %d   ewma only: %d'
      % ((exc_h & exc_e).sum(), (exc_h & ~exc_e).sum(), (~exc_h & exc_e).sum
```

```
Date         2020  2021  2022  2023  2024  2025  2026
historical      8     6    20     0     7    11     3
ewma            7    12    19    13    18    15    10
days          151   252   251   250   252   250    98

mean VaR level: historical 0.0188   ewma 0.0162
days both exceed: 47   historical only: 8   ewma only: 47
```

In [7]:
```
fig, ax = plt.subplots(2, 1, figsize=(11, 7.5), sharex=True)
for a, v, e, name in [(ax[0], var_h, exc_h, 'historical simulation, M = 1010
                      (ax[1], var_e, exc_e, 'normal log returns, EWMA(0.97)'
    a.plot(eval_dates, realized, color='grey', lw=0.5, alpha=0.7, label='rea
    a.plot(eval_dates, v, color='C1', lw=1.2, label=r'VaR$_{0.95}$')
    a.scatter(eval_dates[e], realized[e], color='red', s=12, zorder=3, label
    a.set_title(name)
    a.set_ylabel('loss on $1')
    a.legend(loc='upper left', fontsize=8)
    a.grid(alpha=0.3)
fig.tight_layout()
plt.show()
```

